Zenith: evaluating revisions to a reasoning graph

Will Cook • 12 October 2026

This is not financial advice. This paper describes experimental research. Historical examples and model outputs are not investment recommendations and must not be used to make financial decisions.

Abstract. Zenith studies whether recurring failures in a multi-stage language-model program can guide useful revisions. It records intermediate outputs, diagnoses errors and compares one revision with the original on later inputs. Four historical-feed runs exposed the same truncated response. One instruction change passed local validation on two later inputs, but execution then failed downstream. We report this limited result and propose a controller that selects small prompt variations from market context.

1. Evaluation protocol. Nodes read supplied evidence or earlier outputs and produce records for later nodes. **Lab** selects assets and forecasts from a saved information cutoff. **Oracle** compares the frozen forecast with later observations and proposes a diagnosis; **Evolve** proposes an instruction, context or dependency change. The base language model stays fixed during comparison.

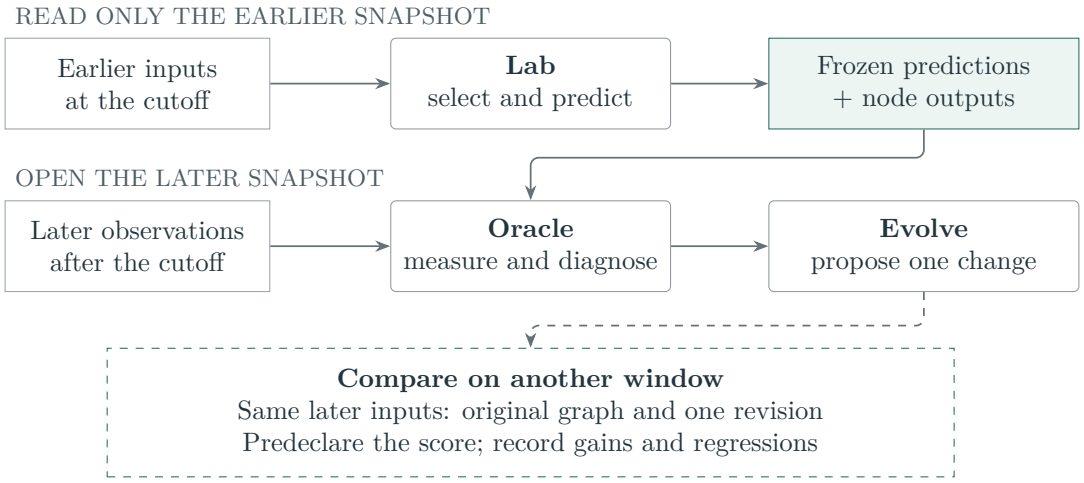


Figure 1. Proposed evaluation loop. Preserve the original record before revealing the outcome. The dashed step is a separate comparison, not evidence that a diagnosis or revision is correct.

Collect runs with the graph held fixed. Group failures across sliding windows and several widths, counting unique cases rather than overlapping windows. Freeze one revision and compare it with the baseline on matched later cases. Cases used to select a revision become development data for subsequent work.

Data and cutoffs. The archive contains stocks, ETFs, macro series, Stockgrid, prediction markets and news, plus calculator summaries. The July 20–21 replay retains all seven artifacts [1]. A packet may contain a current state or dated histories: a "video so far" at the cutoff. Predicting a later state and predicting a path require separate targets and scores. Future proprietary snapshots need verified availability times and revision vintages. Historical inputs alone cannot exclude later knowledge in a current model's training data.

Prior work. DSPy composes LM modules [7]; MIPRO optimises their instructions and examples against a downstream metric [8]. These are direct precedents for prompt optimisation. AlphaGo Zero [4] and autoresearch [5] motivated successive version comparisons. Zenith uses finite external histories; its execution study establishes no forecasting advantage.

2. The executable graph and failure diagnosis. The Lab has 28 nodes: seven feed artifacts and 21 reasoning operators. Each miner is a feed-specific analysis step. Saved intermediate records let a reviewer locate an unsupported claim and inspect the evidence it received.

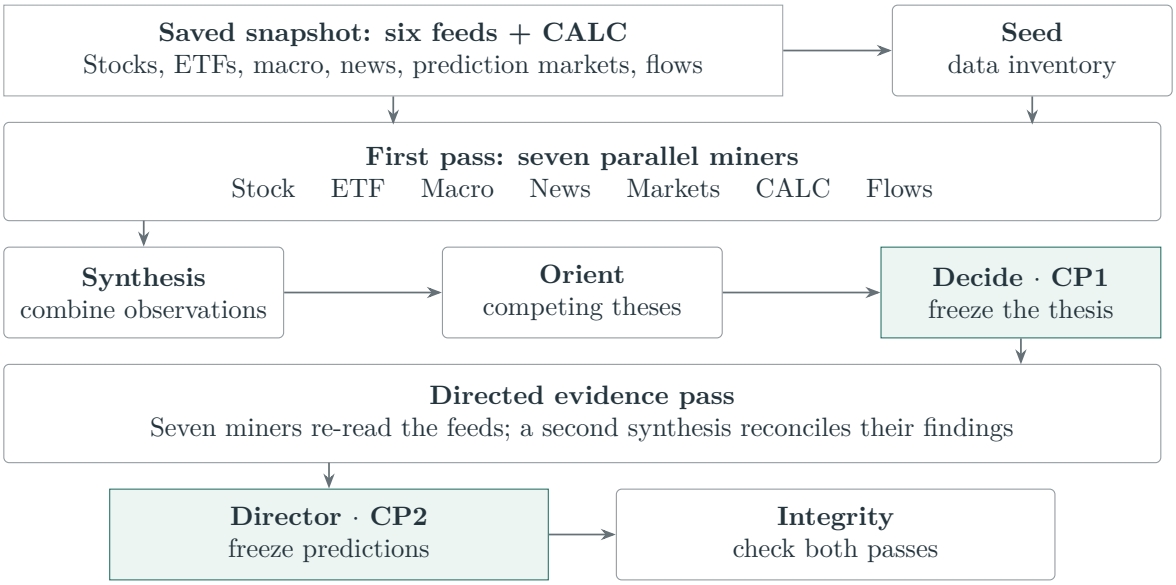


Figure 2. Lab's two passes. "Synthesis" denotes the nodes named Cross-correlation in the implementation; the second synthesis is grouped with the directed pass. CP1 is the saved thesis; in the quantile branch it also freezes forecast quantiles, which Director must preserve at CP2.

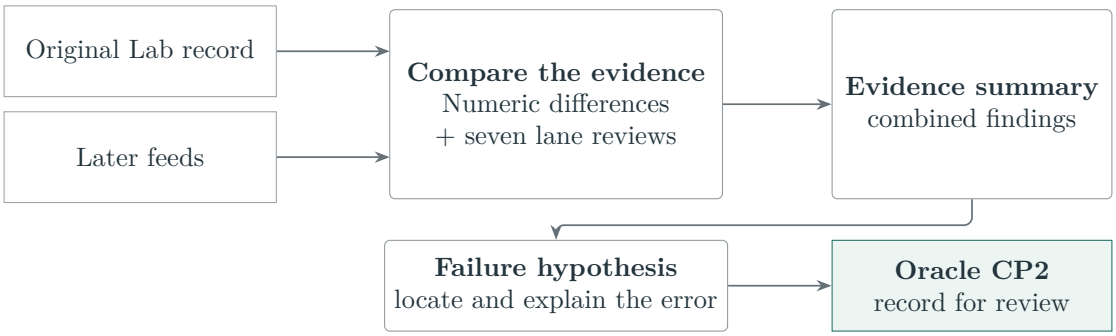


Figure 3. Oracle reviews the saved run against later evidence. Its attribution is a hypothesis about a failure, not a causal identification. Square boxes are records; rounded boxes are operations or groups. Exact feed, skip and cross-pass dependencies are retained in `docs/graphs.md`.

Diagnosis first asks what information was available:

Candidate failure	Question for the saved record
Evidence unavailable	Was the relevant information absent from the saved input universe?
Evidence not supplied	Was it in that universe but missing from the node's actual input?
Reasoning error	Did the node receive the evidence but draw an unsupported conclusion?
Execution failure	Did truncation, parsing or another runtime error prevent a usable output?

Keep unresolved cases unknown. Research performed after the outcome can help explain an event, but cannot become evidence the earlier run supposedly had. A repeated failure motivates an intervention test; frequency alone does not identify its cause.

3. Proposed context-conditioned prompts. Each reasoning node would keep a fixed task, input scope, output contract and work budget. A shared neural controller would read market-condition features available at the cutoff and select small wording variations for that node. We call its outputs *prompt settings*: numbers in $[0, 1]$ that index an ordered catalogue of text variants. They are distinct from the controller's learned numerical weights. This extension has not been implemented or tested.

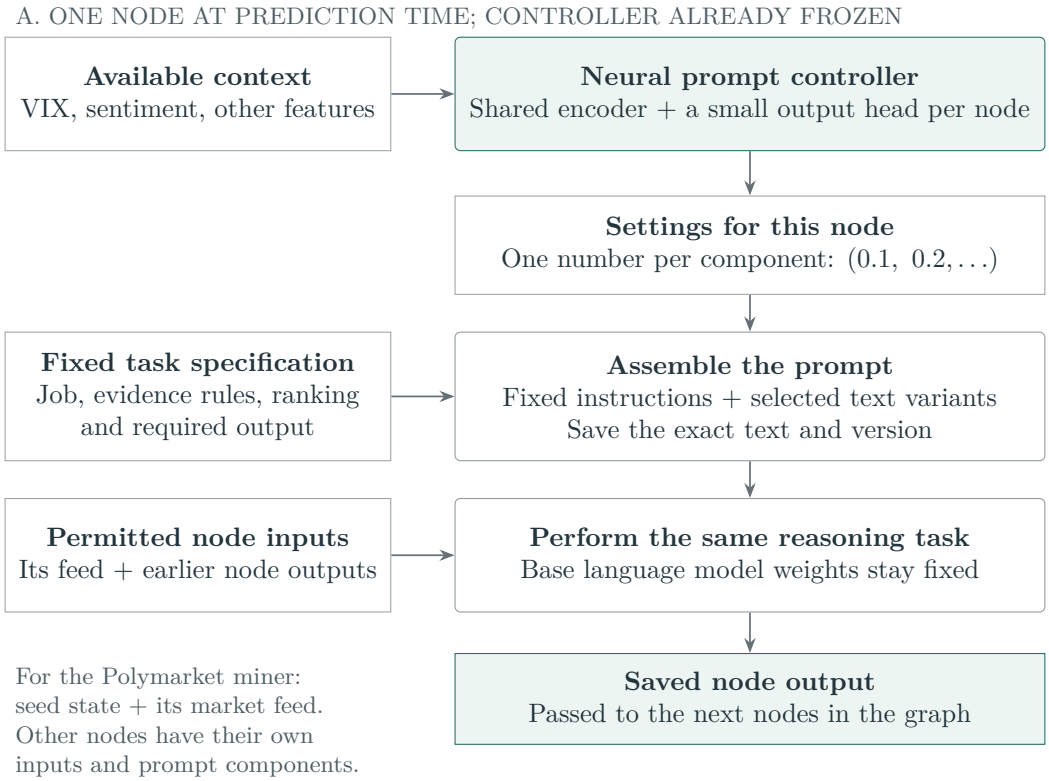
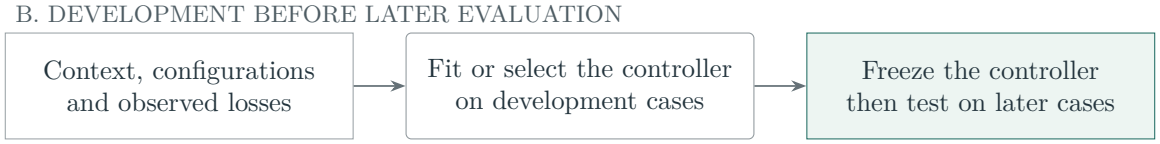


Figure 4. Settings select text; they do not multiply evidence or directly scale the model's reasoning. For example, 0.1 could select "consider sentiment as one explanation" and 0.2 "give sentiment slightly more interpretive weight where supported". Factual filters and ranking rules stay fixed. Nearby settings need not produce smoothly varying behaviour.



A first implementation could search a small catalogue, following discrete prompt optimisation [8]. A neural controller could instead learn from measured configuration losses. Backpropagation would train that controller; hard text selection and hosted LM calls provide no ordinary end-to-end derivative. ProTeGi [9] and TextGrad [10] use verbal feedback as a gradient analogy. Such feedback can suggest a revision, but is neither a numerical derivative nor proof of its benefit. VIX and sentiment are incomplete context features, not a complete regime classification. Later test outcomes must remain outside development.

4. Execution study and limitations. Four full Lab attempts used original snapshots from June 26, July 13, July 20 and July 21, 2026. The graph, policy, full feed data and 4,096-token response limit were fixed, using DeepSeek V4.1 Flash through Deepinfra. Each completed Seed and seven miners, then failed when the first synthesis response ran out of tokens before finishing its JSON [2].

After inspecting the two earlier episodes, we froze one added instruction: preserve all four required arrays, with at most three records per array, four subjects per record and short summaries. The changed node received the same saved upstream outputs on each later input:

Later input	Original instruction	Changed instruction
20 July close	truncated at 4,096	valid JSON, 1,308 tokens
21 July open capture	truncated at 4,096	valid JSON, 1,172 tokens

Both changed responses passed the native schema but violated the new four-subject limit: their largest records contained nine and eleven subjects. Thus local validity improved from 0/2 to 2/2, with incomplete instruction compliance. This comparison measures execution, not forecast accuracy.

A separate continuation reused the eight unchanged July 20 ancestor outputs and all seven feeds. Synthesis and Orient passed; Decide truncated. Those three calls produced no completed CP1, CP2 or forecast [3].

Limits of the comparison. The later windows overlap, the analyst had seen the July 20 failure, and each arm has one model sample. Collection times do not verify underlying price-bar or news clocks; current-model training leakage is unresolved. An agent authored the repair. The full Oracle/Evolve loop has not run, and these retrospective comparisons are not blind, independent trials.

Next evaluation. Complete Lab, then measure selection coverage, forecast error and cost under fixed rules. Use the same fallback for unselected assets and a predeclared score, such as quantile loss at fixed levels [6]. Compare fixed prompts with the proposed controller on later cases, including regime changes and regressions. These experiments could test robustness across sampled conditions; they cannot establish reliable prediction in every possible regime.

References

[1] Zenith. Original feed replay: source hashes and example observations, 2026.

[2] Zenith. Original-feed window experiment, 12 October 2026.

[3] Zenith. Conditional native Lab continuation, 12 October 2026.

[4] D. Silver et al. Mastering the game of Go without human knowledge. *Nature* 550:354–359, 2017. doi:10.1038/nature24270.

[5] A. Karpathy. *autoresearch*. Source inspected at `228791fb499a` (code).

[6] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *JASA* 102(477):359–378, 2007. doi:10.1198/016214506000001437.

[7] O. Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *ICLR*, 2024. arXiv:2310.03714.

[8] K. Opsahl-Ong et al. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. *EMNLP*, 2024. arXiv:2406.11695v2.

[9] R. Pryzant et al. Automatic Prompt Optimization with "Gradient Descent" and Beam Search. *EMNLP*, 2023. arXiv:2305.03495.

[10] M. Yuksekgonul et al. TextGrad: Automatic "Differentiation" via Text. 2024. arXiv:2406.07496v1.